

> 状态: Ready for Development (2026-09-15; 设计 AI (项目负责人授权), ADR 0013)
> Owner: B; 系统基线: Baseline — 5d11769; 依赖波次: Wave 6
> 唯一公开入口: package:locatemy/features/property_inspection/property_inspection.dart
> 完成定义: 消费者仅凭本契约即可安全续填、创建、管理、比较、软删除和清空自己的实勘; 风险快照只在同地点的完整 Safety 与 Hazard 输入同时可用时原子保存。

本文件是 Property Inspection 唯一的跨 Owner 开发协作契约, 也是同名人类 PDF 的 Markdown 源。它固定公开 Dart 声明、输入约束、typed outcomes、权限 / 副作用、顺序、精确数据边界及联合验收; lib/features/property_inspection/ 内的 Widget、状态管理、Supabase/SQLite/Storage Adapter、压缩、队列、并发、取消、重试、文件拆分和测试组织由 B 决定。产品字段、公式、RLS、Storage policy、RPC SQL 和 migration 仍分别以产品知识库与 Schema Catalog 为源。

0. 固定阅读顺序与四项 Readiness

- 1. 领域词汇、房产实勘产品事实、治安与隐患事实;
- 2. Feature map (Property)、Interface 注册表、FLOW-06;
- 3. Capability Traceability、数据所有权、Schema Catalog、风险登记;
- 4. 本契约; 生成 / 使用人类 PDF 时最后读 ADR 0014 与 handoff。

Readiness	可核查证据	结论
责任与范围	PROP-01-PROP-05 唯一归属 Property; 认证、scope、选点、官方安全与公共隐患各有 Owner	已就绪
契约与消费者	第 2 节含 SHELL-001、LOCATION-001、PRIVACY-001、SAFETY-001、HAZARD-002 完整调用卡; 第 3 节有 PROPERTY-001	已就绪
数据与安全	精确使用 property_inspections、property_inspection_photos、inspection_photos、三项本机对象; owner 与父实勘双重授权	已就绪
验收与风险	第 5 节覆盖 AT-PROP-01-06; RISK-PROP-01、RISK-STORAGE-01、RISK-PROPERTY-01 留作实现 / 集成运行时证据	已就绪

1. 成果、责任与冻结边界

- opened scope 的用户可续填带草稿照片的本机实勘草稿; 在线创建或编辑具有 Map 合法地点的实勘, 管理每份至多 20 张私有静态照片, 查看档案 / 详情, 选择 2-3 份活动实勘作不保存的并排比较。
- 删除先进入可见回收站并保留照片; 恢复取消软删除。确认清空才永久删除当前账户回收站内的元数据和 Storage 文件; 任一阶段 partial 均可重试且不称全成功。
- 创建、坐标变更或显式刷新才并行取得同一地点的 SAFETY-001 和 HAZARD-002; 两者均完整 available 时整体创建 / 替换风险组。普通未变坐标编辑保留既有组; 失败 / partial 保留旧组及时间, 绝不补零、拼接或静默刷新。
- 综合评分只由四项现场评分派生, 不是独立字段、州级安全指数、地点指数、排名或推荐。照片是私有视觉证据, 不读取 / 展示相册原件或 EXIF, 也不提供分享 / 导出。

Owner / 受控边界	负责	不负责	协作
lib/features/property_inspection/ (B)	草稿、实勘、照片、队列、比较、回收站、风险快照时机及 PROPERTY-001	Auth、scope、选点、州解析、安全 / 隐患计算、全局路由	消费第 2 节五项 Interface; 提供第 3 节
Application Shell	opened 门控、档案 / 表单 / 详情 / 比较 / 回收站导航、地图返回任务	Property 资料、照片字节、风险 / 队列	SHELL-001
Map / Location	合法 immutable property 地点及返回式选点	草稿、收藏解释、风险、实勘写入	LOCATION-001
Account Privacy	同账户 opened/closing 及关闭证明	远端实勘或已上传照片删除	PRIVACY-001
Crime / Hazard	同地点的安全快照输入; 附近 pending 数	房产写入、照片、比较、回收站	SAFETY-001、HAZARD-002

2. 必须调用的 Interface 卡

SHELL-001 — 房产导航与组合

提供者: Application Shell; 消费者: Property Inspection; 唯一公开 import: package:locatemy/app/application_shell.dart。

```
dart
abstract interface class ApplicationShell {
  Future<ShellIntentOutcome> submit(ShellIntent intent);
  Future<ShellContributionOutcome> publish(ShellContribution contribution);
}
```

```

}
abstract interface class ShellIntent {}
abstract interface class ShellContribution {}
sealed class ShellIntentOutcome {}
final class ShellIntentAccepted extends ShellIntentOutcome {}
final class ShellAuthenticationRequired extends ShellIntentOutcome {}
final class ShellIntentRejected extends ShellIntentOutcome { final ShellRejectionReason reason; }
sealed class ShellContributionOutcome {}
final class ShellContributionAccepted extends ShellContributionOutcome {}
final class ShellContributionAuthenticationRequired extends ShellContributionOutcome {}
final class ShellContributionRejected extends ShellContributionOutcome { final ShellRejectionReason reason; }
enum ShellRejectionReason { missingInput, staleInput, inapplicableDestination, scopeUnavailable }
``,`

```

Property 从其唯一入口导出 OpenPropertyPortfolioIntent、OpenPropertyEditorIntent、OpenPropertyDetailIntent、OpenPropertyCompartment、OpenPropertyRecycleBinIntent、PickPropertyLocationIntent (均为 ShellIntent) 及 PropertyShellContribution (为 ShellContribution)。intent 只带稳定实勘 / 草稿 ID、当前 PropertyReturnContext 或 Map 返回任务; 不携带照片字节、风险上游 payload、跨账户资料或可变地图对象。

输入约束	输出	/ typed failures	状态与副作用	顺序、权限与最小示例
intent / contribution	属当前 opened scope, 稳定 ID / 返回语境未过期。	accepted、authenticationRequired、rejected(reason);	后二者是 Shell 结果而非保存失败。	Shell 只改门控、导航栈与组合, 不写 Property 数据或重算风险。 accepted 才转场; Map 返回先经 LOCATION-001。await shell.submit(OpenPropertyDetailIntent(id, context));

Fake 场景: fake Shell 回 accepted、authentication-required、stale-input rejected; 后两者保留草稿 / 详情和文字恢复路径, 不写入、上传或丢失返回任务。

LOCATION-001 — 合法不可变房产地点

提供者: Map / Location; 消费者: Property Inspection; 唯一公开 import: package:locatemy/features/map_location/map_location.dart

```

``,`dart
abstract interface class LocationCoordinator {
  Future<LocationSelectionOutcome> select(LocationSelectionRequest request);
  LocationRoleSnapshot read(LocationRole role);
}
enum LocationRole { single, locationA, locationB, property }
final class GeographicPoint { final double latitude; final double longitude; }
final class LocationSelectionRequest { final LocationRole role; final GeographicPoint point; final String? displayName; }
final class ValidLocationReference { final String locationId; final GeographicPoint point; final String? displayName; }
sealed class LocationSelectionOutcome {}
final class LocationSelected extends LocationSelectionOutcome { final LocationRole role; final ValidLocationReference location; }
final class LocationSelectionRejected extends LocationSelectionOutcome { final LocationSelectionFailure failure; }
enum LocationSelectionFailure { invalidCoordinate, outsideMalaysia, sameComparisonPoint, scopeUnavailable }
sealed class LocationRoleSnapshot {}
final class LocationPresent extends LocationRoleSnapshot { final LocationRole role; final ValidLocationReference location; }
final class LocationAbsent extends LocationRoleSnapshot { final LocationRole role; }
``,`

```

LocateMY · Property Inspection 开发协作契约 · 第 3 / 9 页
输入约束	输出 / typed failures	状态与副作用	顺序、权限与最小示例
只接受 Map 已选出的 property 角色成功引用;有限 WGS84 数值且已通过马来西亚范围校验。	selected / present, 或		
invalidCoordinate、outsideMalaysia、scopeUnavailable。 | Map 只改自己的角色;Property 不读可变 Map 状态、不造坐标、不把收藏名当地点。 |
选点返回后才写草稿 / 请求风险;失败保留原草稿地点。 LocationSelected(:final location) => draft.withLocation(location)。 |

Fake 场景: fake Map 给 property selected、absent、outsideMalaysia、scope unavailable;只有 selected 的 immutable 引用可创建、改坐标或刷新风险,其他结果绝不触发 Safety/Hazard。

PRIVACY-001 — 账户范围与 Property close 参与者

提供者: Account Privacy;消费者: Property Inspection;唯一公开 import:
package:locatemy/features/account_privacy/account_privacy.dart。

```
```\ndart
final class AccountScope { final String accountId; const AccountScope(this.accountId); }
sealed class AccountScopeSnapshot {}
final class AccountScopeOpened extends AccountScopeSnapshot { final AccountScope scope; }
final class AccountScopeClosed extends AccountScopeSnapshot { final AccountScope scope; }
final class AccountScopeUnavailable extends AccountScopeSnapshot { final AccountScopeFailure failure; }
enum AccountScopeFailure { identityMismatch, closing, incompleteOwners, retryableUnavailable }
abstract interface class AccountPrivacy {
 AccountScopeSnapshot readScope();
 Future<PropertyPrivateStateCleared> clearPropertyPrivateState(AccountScope scope);
}
sealed class PropertyPrivateStateCleared {}
final class PropertyPrivateStateClearedForAccount extends PropertyPrivateStateCleared { final String accountId; }
final class PropertyPrivateStateClearedIncomplete extends PropertyPrivateStateCleared { final String accountId; final PropertyPrivateStateFailure failure; }
enum PropertyPrivateStateFailure { localStoreUnavailable, fileDeletionIncomplete, queueCleanupIncomplete, scopeUnavailable }
```\n
```

输入约束	输出 / typed failures	状态与副作用	顺序、权限与最小示例
所有私有读写仅同账户 opened;clear 只针对 Privacy 给出的不可变旧 scope。	opened/closed/unavailable;清理为 cleared 或 incomplete,均携带账户与分		
closing 开始即阻断草稿、草稿照片、私有副本、比较、正式待传 queue 和应用目录文件;不删远端实勘、元数据或已上传 Storage。	Shell 发起 close;Property 只报告自身结果。 await		
privacy.clearPropertyPrivateState(oldScope);;incomplete 时旧内容仍不可读且可重试。 |

Fake 场景: A 有草稿 / 上传中照片后 close, fake Privacy 随后开 B;A 的表单、文件、比较和晚到成功均不可显示 / 提交 / 重放, B 只从自己的远端档案开始。

SAFETY-001 — 州级安全快照输入

提供者: Crime & Security;消费者: Property Inspection;唯一公开 import:
package:locatemy/features/crime_and_security/crime_and_security.dart。

```
```\ndart
abstract interface class CrimeAndSecurity { Future<SafetyLoadOutcome> load(SafetyRequest request); }
final class SafetyRequest { final ValidLocationReference location; final SafetyLoadPolicy policy; final SafetyTrendFilter filter; const SafetyRequest({required this.location, required this.policy, required this.filter})
enum SafetyLoadPolicy { cacheAllowed, refresh }
sealed class SafetyTrendFilter { const SafetyTrendFilter(); }
```

```
final class AllCrimeTrend extends SafetyTrendFilter { const AllCrimeTrend(); }
sealed class SafetyLoadOutcome { const SafetyLoadOutcome(); }
final class SafetyAvailable extends SafetyLoadOutcome { final SafetySnapshot snapshot; const
SafetyAvailable(this.snapshot); }
final class SafetyPartiallyAvailable extends SafetyLoadOutcome { final SafetySnapshot snapshot; const
SafetyPartiallyAvailable(this.snapshot); }
final class SafetyUnavailable extends SafetyLoadOutcome { final SafetyUnavailableReason reason; const
SafetyUnavailable(this.reason); }
final class SafetySnapshot { final ValidLocationReference location; final ReportingState state; final SafetyScore
score; final AnnualCrimeCount latestCompleteYearCount; final SafetyFreshness freshness; final SafetyCompleteness
completeness; final SafetyProvenance provenance; const SafetySnapshot({required this.location, required this.stat
required this.score, required this.latestCompleteYearCount, required this.freshness, required this.completeness,
required this.provenance}); }
enum SafetyFreshness { fresh, cached, stale }
enum SafetyCompleteness { complete, partial }
enum SafetyUnavailableReason { stateUnresolved, stateAmbiguous, sourceMissing, noValidCategory, incompleteYear,
retryableUnavailable, sourceUnverifiable }
final class SafetyScore { final int value; const SafetyScore(this.value); }
final class AnnualCrimeCount { final int value; final int year; const AnnualCrimeCount(this.value, this.year); }
final class ReportingState { final String stableId; final String name; const ReportingState(this.stableId, this.n
}
final class SafetyProvenance { final String source; final String modelVersion; final String boundaryVersion; cons
SafetyProvenance(this.source, this.modelVersion, this.boundaryVersion); }
```

| 输入约束 | 输出 / typed failures | 精确数据边界与副作用 | 顺序、权限与最小示例 |

| --- | --- | --- | --- |

| location 必为同一 LOCATION-001 property immutable reference;普通保存用 cacheAllowed, 显式刷新用 refresh。 | 仅 SafetyAvailable 且 completeness complete 可供 snapshot;partial/unavailable 均不可用。 | Crime 只读 read\_safety\_inputs (聚合 crime\_district) 并可读写无 crime\_public\_cache;不写 Property、Hazard、Map 或 Shell。 | 与 Hazard 对同一引用并行;只读 score/state/year/provenance, 绝不自行解析州。 |

Fake 场景: fake 依次给 complete、cached complete、partial、unresolved、retryable;Property 仅采纳 complete 的同地点数据, partial/unavailable 保留既有风险组且不能显示为 0。

HAZARD-002 — 附近 pending 隐患数

提供者: Hazard Reporting;消费者: Property Inspection;唯一公开 import:  
package:locatemy/features/hazard\_reporting/hazard\_reporting.dart。

```
dart
abstract interface class HazardRiskCounter { Future<HazardNearbyCountOutcome> countPending(HazardNearbyCountReque
request); }
final class HazardNearbyCountRequest { final ValidLocationReference propertyLocation; const
HazardNearbyCountRequest(this.propertyLocation); }
sealed class HazardNearbyCountOutcome { const HazardNearbyCountOutcome(); }
final class HazardNearbyCountAvailable extends HazardNearbyCountOutcome { final int count; final int radiusMeters
final DateTime countedAt; const HazardNearbyCountAvailable(this.count, this.radiusMeters, this.countedAt); }
final class HazardNearbyCountUnavailable extends HazardNearbyCountOutcome { final HazardNearbyCountFailure failure
const HazardNearbyCountUnavailable(this.failure); }
enum HazardNearbyCountFailure { invalidLocation, authenticationRequired, partialResult, retryableUnavailable,
scopeUnavailable }
```

| 输入约束 | 输出 / typed failures | 精确数据边界与副作用 | 顺序、权限与最小示例 |

| --- | --- | --- | --- |

| propertyLocation 是同一 Map immutable reference。 | available 带 count/radiusMeters/countedAt;unavailable 不得替换为 0。 |

Hazard 只读 crowdsourced\_hazards;Haversine d <= 2,000m (含边界), 仅 pending;不写 reports、votes、hazard\_vote\_counts RPC、图层或 Property。 | 同地点 Safety 并行后才决定原子写。 await counter.countPending(HazardNearbyCountRequest(location)); |

Fake 场景: fake 给内/外/恰 2,000m、pending/resolved、partial;只计内/边界 pending, partial/failure 不生成 0 或新快照。

### 3. 必须提供的 Interface

PROPERTY-001 — 私有实勘、照片、比较、回收站与风险快照

提供者: Property Inspection (B);消费者: Application Shell;唯一公开 import:  
package:locatemy/features/property\_inspection/property\_inspection.dart。

消费者只能 import 此入口。B 应先合入下列声明与最小 fake;这是协作形状, 不是可提交实现体、SQL、Storage policy 或 SDK 映射。

```
dart
abstract interface class PropertyInspection {
 Future<PropertyPortfolioOutcome> loadPortfolio();
 Future<PropertyDraftOutcome> loadDraft(PropertyDraftId id);
 Future<PropertyDraftOutcome> saveDraft(PropertyDraft draft);
 Future<PropertyWriteOutcome> create(PropertyWriteRequest request);
 Future<PropertyWriteOutcome> update(PropertyInspectionId id, PropertyWriteRequest request);
 Future<PropertyDetailOutcome> loadDetail(PropertyInspectionId id);
 Future<PropertyPhotoOutcome> addPhoto(PropertyPhotoRequest request);
 Future<PropertyPhotoOutcome> updatePhoto(PropertyPhotoUpdate request);
 Future<PropertyPhotoOutcome> deletePhoto(PropertyPhotoId id);
 Future<PropertyComparisonOutcome> compare(List<PropertyInspectionId> ids);
 Future<PropertyRecycleBinOutcome> loadRecycleBin();
 Future<PropertyDeleteOutcome> softDelete(PropertyInspectionId id);
 Future<PropertyRestoreOutcome> restore(PropertyInspectionId id);
 Future<PropertyPurgeOutcome> purgeRecycleBin(PropertyPurgeConfirmation confirmation);
 Future<PropertyRiskRefreshOutcome> refreshRisk(PropertyInspectionId id);
}

final class PropertyInspectionId { final String value; const PropertyInspectionId(this.value); }
final class PropertyDraftId { final String value; const PropertyDraftId(this.value); }
final class PropertyPhotoId { final String value; const PropertyPhotoId(this.value); }
final class PropertyWriteRequest { final String name; final String address; final ValidLocationReference location
final String? savedLocationId; final String price; final int drainage; final int waterproofing; final int humidit
final int lighting; final bool floodEvidence; final String? notes; const PropertyWriteRequest({required this.name
required this.address, required this.location, this.savedLocationId, required this.price, required this.drainage,
required this.waterproofing, required this.humidity, required this.lighting, required this.floodEvidence,
this.notes}); }
final class PropertyDraft { final PropertyDraftId id; final PropertyWriteRequest values; final DateTime updatedAt
const PropertyDraft(this.id, this.values, this.updatedAt); }
final class PropertyPhotoRequest { final PropertyInspectionId? inspectionId; final PropertyDraftId? draftId; fina
String localPhotoReference; final String? caption; const PropertyPhotoRequest({this.inspectionId, this.draftId,
required this.localPhotoReference, this.caption}); }
final class PropertyPhotoUpdate { final PropertyPhotoId id; final String? caption; final bool? cover; const
PropertyPhotoUpdate(this.id, {this.caption, this.cover}); }
```

```
final class PropertyPurgeConfirmation { final bool confirmed; const PropertyPurgeConfirmation(this.confirmed); }
sealed class PropertyPortfolioOutcome { const PropertyPortfolioOutcome(); }
final class PropertyPortfolioAvailable extends PropertyPortfolioOutcome { final List<PropertySummary> inspections
const PropertyPortfolioAvailable(this.inspections); }
final class PropertyPortfolioUnavailable extends PropertyPortfolioOutcome { final PropertyReadFailure failure; co
PropertyPortfolioUnavailable(this.failure); }
sealed class PropertyDraftOutcome { const PropertyDraftOutcome(); }
final class PropertyDraftAvailable extends PropertyDraftOutcome { final PropertyDraft draft; const
PropertyDraftAvailable(this.draft); }
final class PropertyDraftAbsent extends PropertyDraftOutcome { const PropertyDraftAbsent(); }
final class PropertyDraftRejected extends PropertyDraftOutcome { final PropertyWriteFailure failure; const
PropertyDraftRejected(this.failure); }
sealed class PropertyWriteOutcome { const PropertyWriteOutcome(); }
final class PropertyWritten extends PropertyWriteOutcome { final PropertyDetail inspection; final
PropertyPhotoTransferState photoTransfer; const PropertyWritten(this.inspection, this.photoTransfer); }
final class PropertyWriteRejected extends PropertyWriteOutcome { final PropertyWriteFailure failure; const
PropertyWriteRejected(this.failure); }
sealed class PropertyDetailOutcome { const PropertyDetailOutcome(); }
final class PropertyDetailAvailable extends PropertyDetailOutcome { final PropertyDetail inspection; const
PropertyDetailAvailable(this.inspection); }
final class PropertyDetailUnavailable extends PropertyDetailOutcome { final PropertyReadFailure failure; const
PropertyDetailUnavailable(this.failure); }
sealed class PropertyPhotoOutcome { const PropertyPhotoOutcome(); }
final class PropertyPhotoChanged extends PropertyPhotoOutcome { final PropertyPhoto photo; const
PropertyPhotoChanged(this.photo); }
final class PropertyPhotoRejected extends PropertyPhotoOutcome { final PropertyPhotoFailure failure; const
PropertyPhotoRejected(this.failure); }
sealed class PropertyComparisonOutcome { const PropertyComparisonOutcome(); }
final class PropertyComparisonAvailable extends PropertyComparisonOutcome { final List<PropertyDetail> inspection
const PropertyComparisonAvailable(this.inspections); }
final class PropertyComparisonRejected extends PropertyComparisonOutcome { final PropertyComparisonFailure failure
const PropertyComparisonRejected(this.failure); }
sealed class PropertyRecycleBinOutcome { const PropertyRecycleBinOutcome(); }
final class PropertyRecycleBinAvailable extends PropertyRecycleBinOutcome { final List<PropertySummary> inspectio
const PropertyRecycleBinAvailable(this.inspections); }
final class PropertyRecycleBinUnavailable extends PropertyRecycleBinOutcome { final PropertyReadFailure failure;
PropertyRecycleBinUnavailable(this.failure); }
sealed class PropertyDeleteOutcome { const PropertyDeleteOutcome(); }
final class PropertySoftDeleted extends PropertyDeleteOutcome { final PropertyInspectionId id; const
PropertySoftDeleted(this.id); }
final class PropertyDeleteRejected extends PropertyDeleteOutcome { final PropertyWriteFailure failure; const
PropertyDeleteRejected(this.failure); }
sealed class PropertyRestoreOutcome { const PropertyRestoreOutcome(); }
final class PropertyRestored extends PropertyRestoreOutcome { final PropertyInspectionId id; const
PropertyRestored(this.id); }
final class PropertyRestoreRejected extends PropertyRestoreOutcome { final PropertyRestoreRejectedFailure failure
const PropertyRestoreRejected(this.failure); }
sealed class PropertyPurgeOutcome { const PropertyPurgeOutcome(); }
final class PropertyPurgeCancelled extends PropertyPurgeOutcome { const PropertyPurgeCancelled(); }
final class PropertyPurgeCompleted extends PropertyPurgeOutcome { const PropertyPurgeCompleted(); }
final class PropertyPurgePartial extends PropertyPurgeOutcome { final List<PropertyInspectionId> remaining; final
PropertyPurgeFailure failure; const PropertyPurgePartial(this.remaining, this.failure); }
```

```
sealed class PropertyRiskRefreshOutcome { const PropertyRiskRefreshOutcome(); }
final class PropertyRiskRefreshed extends PropertyRiskRefreshOutcome { final PropertyRiskSnapshot snapshot; const
PropertyRiskRefreshed(this.snapshot); }
final class PropertyRiskRefreshUnavailable extends PropertyRiskRefreshOutcome { final PropertyRiskFailure failure
const PropertyRiskRefreshUnavailable(this.failure); }
enum PropertyReadFailure { authenticationRequired, notFound, permissionDenied, retryableUnavailable, scopeUnavail
}
enum PropertyWriteFailure { invalidName, invalidAddress, invalidLocation, invalidPrice, invalidScore, invalidNote
onlineRequired, permissionDenied, conflict, notFound, retryableUnavailable, scopeUnavailable, riskUnavailable }
enum PropertyPhotoFailure { limitReached, invalidPhotoSource, unsupportedFormat, devicePermissionDenied,
compressionFailed, onlineDeleteRequired, permissionDenied, parentNotFound, retryableUnavailable, transferIncomple
scopeUnavailable }
enum PropertyComparisonFailure { fewerThanTwo, moreThanThree, deletedOrInvisible, retryableUnavailable,
scopeUnavailable }
enum PropertyRestoreRejectedFailure { permissionDenied, notFound, conflict, retryableUnavailable, scopeUnavailabl
enum PropertyPurgeFailure { metadataDeletionIncomplete, storageDeletionIncomplete, retryableUnavailable,
permissionDenied, scopeUnavailable }
enum PropertyRiskFailure { safetyUnavailable, hazardUnavailable, partialInput, locationMismatch, retryableUnavail
scopeUnavailable }
enum PropertyPhotoTransferState { none, pendingUpload, transferIncomplete }
final class PropertySummary { final PropertyInspectionId id; final String name; final String address; final int
derivedOverallScore; final PropertyPhoto? cover; const PropertySummary(this.id, this.name, this.address,
this.derivedOverallScore, this.cover); }
final class PropertyPhoto { final PropertyPhotoId id; final String? caption; final bool isCover; final
PropertyPhotoTransferState transfer; const PropertyPhoto(this.id, this.caption, this.isCover, this.transfer); }
final class PropertyRiskSnapshot { final ReportingState reportingState; final int safetyIndex; final int
safetySourceYear; final String safetySourceId; final String safetyModelBoundaryVersion; final int hazardPendingCo
final int hazardRadiusMeters; final DateTime hazardCountedAt; final DateTime capturedAt; const
PropertyRiskSnapshot({required this.reportingState, required this.safetyIndex, required this.safetySourceYear,
required this.safetySourceId, required this.safetyModelBoundaryVersion, required this.hazardPendingCount, require
this.hazardRadiusMeters, required this.hazardCountedAt, required this.capturedAt}); }
final class PropertyDetail { final PropertySummary summary; final List<PropertyPhoto> photos; final
PropertyRiskSnapshot? riskSnapshot; const PropertyDetail(this.summary, this.photos, this.riskSnapshot); }
```

| 调用 | 输入约束 | typed output / failure | 状态、副作用、权限与顺序 |

| --- | --- | --- | --- |

| loadPortfolio/loadDetail/loadRecycleBin | opened; 稳定 ID 属当前账户。 | available、明确空列表、notFound/permission/retryable/scope; 空不代表失败。 | 精确读 property\_inspections: 活动 deleted\_at 缺失, 回收站 delet

存在; owner-only。详情读其 property\_inspection\_photos。 |

| loadDraft/saveDraft | opened; draft 属账户; 草稿可保留完整表单及草稿照片引用。 | available/absent/rejected。 | 精确读写 SQLite

property\_drafts、property\_draft\_photos; 跨重后而非远端创建/发布; close 必清。 |

| create/update | opened、在线; name trim 1-200、必需 WGS84 location、非负价格、四项 1-5、notes/收藏引用遵守 Catalog。 | written 或

validation、online-required、permission/conflict/not-found/retryable/risk failure。 | 写 property\_inspections

owner-only。create/坐标变更只在两完整输入时原子写风险组; 未变坐标 update 保留组。成功 create 后草稿照片整体转入 SQLite

property\_photo\_upload\_queue; 所有转换成功前保留草稿对象。 |

| addPhoto/updatePhoto/deletePhoto | opened; 仅一项 inspectionId 或 draftId; 常见静态格式、压缩后、≤20、caption ≤1000; 单张删除在线。 | cha

或 limit/source/format/device/compression/online/parent/transfer failure。 | 草稿照片写 property\_draft\_photos; 正式 metadata

property\_inspection\_photos, 文件写私有 bucket inspection-photos 路径 accountId/inspectionId/photoId。读写删均验证账户与父实勘

owner; 上传/metadata 非原子时保留 queue/local copy; 封面删除回退至最早剩余。 |

| compare | opened; 正好 2-3 个不同、活动、可见、同账户 ID。 | available 或 fewer/more/deleted-or-invisible/read/scope failure。 | 只

STATE-PROPERTY-COMPARE; 不写任何表, 不混 Map A/B, 不产生赢家。 |

LocateMY · Property Inspection 开发协作契约 · 第 8/9 页

| softDelete/restore/purgeRecycleBin | opened; 删除/清空均经用户确认; purge confirmation 明确。 | deleted/restored; cancelled/completed/partial(remaining)。 | delete/restore 只改 property inspections.deleted\_at, 不删照片; purge 只删除当前账户回收站中的 Storage 文件、metadata、记录, 逐项 partial 可重试、已删幂等。 | refreshRisk | opened、在线; 详情的同一 immutable location。 | refreshed 或 safety/hazard/partial/mismatch/retryable/scope unavailable。 | 并行 Safety/Hazard; 仅完整且同 location 的结果整体替换 Catalog 的风险字段组。失败保留旧组与时间。 |

最小调用：

```
dart
final written = await properties.create(request);
switch (written) {
 case PropertyWritten(:final inspection, :final photoTransfer):
 // 只显示权威 inspection;pending/transferIncomplete 仍是可重试状态。
 case PropertyWriteRejected(:final failure):
 // 保留草稿及草稿照片, 按 typed failure 给出修正或联网路径。
}
```

Fake 场景: Shell 使用 fake PropertyInspection 分别回空档案、草稿、written + pending upload、风险 unavailable、照片 transfer incomplete、比较不足/越界、purge partial; 只有明确成功才改变呈现, 失败不丢草稿、不把 pending 称发布、也不把 partial purge 称清空。

4. 数据、固定顺序与安全不变量

| 目的 | 权威对象 / 精确访问 | 固定语义 |

| --- | --- | --- |

| 实勘和风险组 | property inspections (owner-only CRUD) | 必需地点、名称/价格/四评分/水灾/备注/收藏引用及原子风险字段组均以 Catalog 为准。deleted\_at 分隔活动/回收站; 软删除不删照片。 |

| 正式照片 | property inspection\_photos (owner + 父实勘授权); Storage inspection-photos (accountId/inspectionId/photoId) | 元数据和文件独立。每实勘最多 20、唯一封面、caption ≤1000; 路径校验不能替代父对象授权。 |

| 草稿与队列 | SQLite property\_drafts、property\_draft\_photos、property\_photo\_upload\_queue 及应用目录 | 全部带 account 分区; 草稿/草稿照片跨重后而未发布; queue 只接收已成功创建的同账户远端实勘照片; 退出/换号清除。 |

| Safety 输入 | SAFETY-001; 其受控读取为 read\_safety\_inputs / crime\_district 与无账户 crime\_public\_cache | Property 只消费同地点 com.SafetyAvailable 的州、分数、年、来源、模型/边界版本; 不读原始行、不解析州。 |

| Hazard 输入 | HAZARD-002; 其读取为 crowdsourced\_hazards | d <= 2,000m、仅 public pending、带 count 时间/半径; 不读/写 vote 或 hazard\_vote\_counts RPC。 |

固定顺序: 1) Shell 只在 opened scope 启动 Property; 2) Map 返回合法 immutable property location; 3) Property 保存草稿; 4) create、坐标变更或刷新同时请求 Safety/Hazard; 5) 两项均 complete available 且 location 相同才在同一远端写操作整体写风险组; 6) create 成功才把草稿照片整体转换为正式 queue, 所有转换成功才移除草稿照片; 7) 正式照片在 Storage 上传和 metadata 成功后才移除本机副本; 8) close 先阻断、后清理本机私有状态, 绝不重放至新账户。

5. 联合验收、自由度与完成核对

| Capability / canonical AT | 场景与操作 | 可观察完成条件 |

| --- | --- | --- |

| PROP-01 / AT-PROP-01 | 新增/编辑、草稿、草稿照片、重后、Map 往返、在线/离线 | 当前账户草稿完整续填; 地点/字段/online/risk/conflict 分开呈现; 合法地点与完整风险输入才创建/变更坐标, 失败保留修正/重试路径。 |

| PROP-02 / AT-PROP-02 | 相机/相册、20 张、封面、说明、草稿转正式、重后、部分上传 | 草稿照片未发布; 所有转换成功前保留草稿; 正式 queue / 本机副本重试正确, 原件/EXIF 不泄露, 离线单张删除不伪成功。 |

| PROP-03 / AT-PROP-03 | 创建、变坐标、未变坐标编辑、刷新、Safety/Hazard complete/partial/失败 | 只在两完整同地点输入时整体保存/替换风险组; 失败/partial 不改旧组或时间, 显示本次不可用。 |

| PROP-01-03 / AT-PROP-04 | A/B 账户、伪造 inspection ID/Storage path、close 中晚到结果 | 仅 owner 且父对象授权可访问; 无地点拒绝; 旧范围资料、上传和结果不显示/提交/重放。 |

LocateMY · Property Inspection 开发协作契约 · 第 9 / 9 页

| PROP-04 / AT-PROP-05 | 2、3、少于 2、超过 3、已删除 / 他人项 | 只有 2-3 个当前账户活动实勘并排；比较不保存、不混 Map A / B、没有赢家。 |

| PROP-05 / AT-PROP-06 | 软删除、取消、恢复、确认清空、Storage / metadata partial | 回收站可见且软删除保留照片；取消零副作用；清空仅当前账户，partial 列明 remain 并可重试，不误报全成功。 |

| 全部 | 中文 / English、200% 字体、读屏 / 非颜色状态 | 字段错误、同步 / 删除、风险来源 / 时间 / 口径、空态、权限和恢复路径均有文字等价信息。 |

Owner 可选择内部架构、数据库 / Storage SDK 映射、压缩、上传调度、缓存、重试、取消、并发、Widget 与测试组织。以下变更须取得跨 Owner agreement：唯一公开 import、任一公开声明 / res variant、输入约束、风险原子性、账户 / 父对象授权、Storage 路径契约或 Schema / RLS。变更同时更新契约、受影响 fake / Adapter 测试和 PDF。

- [x] 按固定顺序完成成果、责任、调用 Interface、提供 Interface、精确数据、实现顺序、联合验收、自由度 / 参考。
- [x] SHELL-001、LOCATION-001、PRIVACY-001、SAFETY-001、HAZARD-002 和 PROPERTY-001 均有唯一 import、声明 / 精确调用、约束、typed outcome、权限 / 副作用、顺序、示例与 fake。
- [x] 草稿、照片队列、风险原子写、账户隔离、回收站和 Storage partial 均有可观察结果；字段、RLS、路径、RPC SQL、migration 和公式仍有唯一外部权威。
- [x] Standards / Spec 双轴复审通过；无 Ready 阻塞。风险的 migration / RLS / Storage / 故障注入证据留实现与集成验收。

| 日期 | 状态 | 变更原因 | 受影响对象 | 批准者 |

| --- | --- | --- | --- | --- |

| 2026-09-15 | Ready for Development | Issue #23：升级为单一契约优先 Markdown / PDF；补齐固定顺序、Readiness、五项上游调用卡与 PROPERTY-001 声明，不改变产品、Schema 或上游语义 | PROP-01-05、PROPERTY-001、SHELL-001、LOCATION-001、PRIVACY-001、SAFETY-001、HAZARD-002、Property | 设计 AI（项目负责人授权） |